Task async await və paralel işin məntiqi

`Task` əməliyyatın tamamlanmasını və nəticəsini təmsil edir. `async` metodu asinxron gözləmələr yazmağa hazırlayır. `await` isə uyğun əməliyyatın tamamlanmasını cari thread-i bloklamadan gözləməyə imkan verir. Bu üç anlayışdan heç biri təkbaşına “mütləq yeni thread açılır” demək deyil.

İlkin `Sinxron Proqramlaşdırma.pdf` və `files/01_dotnet_core_fundamentals.pdf` materiallarında async-in ümumi izahı var idi. Task-ın həmişə ThreadPool üzərində işləməsi, await-dən sonra mütləq eyni thread-ə qayıtması və ValueTask-ın həmişə stack-də olması kimi ümumiləşdirmələr burada düzəldilir. Cancellation, exception, məhdud paralellik və context ömrü də əlavə olunur.

Mündəricat

Task async await və paralel işin məntiqi **1**

1 Proses thread concurrency parallelism . . . 2

2 Task əməliyyatın vədidir thread deyil . . . 2

3 Async metod ilk haraya qədər işləyir . . . 2

4 I/O və CPU üçün fərqli seçim . . . 3

5 Ardıcıl await və birlikdə başlatma . . . 3

6 WhenAll xətası və WhenAny . . . 4

7 CancellationToken və vaxt sərhədi . . . 4

8 Result Wait və deadlock . . . 5

9 Async void və resurs ömrü . . . 5

10 ValueTask nə vaxt lazımdır . . . 5

11 Race condition və məhdud paralellik . . . 6

12 Müsahibə cavabları və məşq . . . 6

13 Rəsmi mənbələr . . . 7

1 Proses thread concurrency parallelism

Process işləyən proqramın əməliyyat sistemi mühitidir. **Thread** onun kod icra edən axınıdır. **CPU core** prosessorun hesablama aparan nüvəsidir. **ThreadPool** .NET-in qısa işlər üçün idarə etdiyi worker thread-lər mexanizmidir. Hər iş üçün sıfırdan thread yaratmaq əvəzinə mövcud resurslardan istifadə edir.

Concurrency, eyni vaxtlılıq, bir neçə işin vaxtları üst-üstə düşən şəkildə irəliləməsidir. Bir nüvədə növbələşmə və I/O gözləməsi ilə də mümkündür. **Parallelism**, paralellik, müxtəlif iş hissələrinin həqiqətən eyni anda icrasıdır. **Asynchrony**, asinxronluq, işin tamamlanmasını gözləyərkən çağıran icra axınının bloklanmamasına imkan verən modeldir. Bu anlayışlar əlaqəlidir, amma sinonim deyil.

I/O, Input Output, disk, şəbəkə və başqa xarici mənbələrlə məlumat mübadiləsidir. **I/O-bound** işdə əsas vaxt cavabı gözləməyə gedir. **CPU-bound** işdə əsas vaxt hesablamaya sərf olunur. Şəkli hesablamaq CPU, şəkli serverdən yükləmək I/O üstünlüklü ola bilər. Real əməliyyat hər iki mərhələdən ibarət ola bilər.

Latency bir əməliyyatın cavab müddətidir. **Throughput** müəyyən vaxtda tamamlanan iş sayıdır. Async serverdə eyni gözləmə müddətində daha çox sorğuya xidmət etməyə kömək edə bilər; bir uzaq serverin 500 ms cavabını avtomatik 50 ms etmir.

2 Task əməliyyatın vədidir thread deyil

`Task` nəticə dəyəri olmayan tamamlanmanı, `Task<T>` isə T tipli nəticəsi olan tamamlanmanı təmsil edir. Məsələn, `Task<int>` sonradan bir tam ədəd verəcək əməliyyatdır. Task nəticələnə bilər, xəta ilə bitə bilər və ya ləğv oluna bilər. **Faulted** xəta ilə bitmiş, **Canceled** ləğv olunmuş, **RanToCompletion** uğurla tamamlanmış vəziyyətdir.

`Task.Run` CPU işini adətən ThreadPool-da planlayır. Amma `HttpClient.GetStringAsync` ilə şəbəkə cavabı gözlənərkən həmin Task üçün ayrıca thread-in bütün vaxt boyu tutulması tələb olunmur. Əməliyyat sistemi I/O tamamlanma bildirişini verəndə kodun davamı planlaşdırıla bilər. `Task.CompletedTask` artıq bitmiş Task-dır; ona xüsusi işçi thread lazım deyil.

Scheduler işin harada və nə vaxt icra ediləcəyini təşkil edən mexanizmdir. **Continuation** gözləmədən sonra işləyəcək davam kodudur. Task və continuation-ların sayı thread sayına bərabər deyil. 1000 açıq HTTP gözləməsi avtomatik 1000 işçi thread demək deyil.

3 Async metod ilk haraya qədər işləyir

C#

```
static async Task DemoAsync()
{
    Console.WriteLine("A");
    await Task.Delay(1000);
    Console.WriteLine("B");
}

Task work = DemoAsync();
Console.WriteLine("C");
await work;
```

`DemoAsync` çağırılan thread-də başlayır. A yazılır. Delay hələ tamamlanmadığı üçün metod gözləmə nöqtəsində çağırana qayıdır. C yazılır. Gecikmə tamamlanandan sonra B yazılır. Bu nümunədə gözlənən sıra A, C, B-dir. `Task.Delay` timer, vaxt bildirişi, ilə tamamlanan gözləmədir; `Thread.Sleep` kimi thread-i həmin müddətdə bloklamır.

Await olunan əməliyyat artıq bitibsə metod dərhal davam edə bilər və çağırana gözləmə üçün qayıtmaz. Ona görə “hər await thread-i buraxır və dəyişir” cümləsi doğru deyil. Async metodun await-ə qədər olan ağır CPU dövrü də çağıran thread-də işləyir. Tək `async` sözünü əlavə etməklə CPU işi arxa plana keçmir.

State machine metodun hansı gözləmə mərhələsində olduğunu və lazım olan lokal dəyərləri saxlayan vəziyyət mexanizmidir. Kompilyator async metodu buna uyğun çevirir. Daxili state rəqəmləri compiler təfərrüatıdır; “0 həmişə başlanğıcdır, -1 həmişə bitdi” kimi universal cədvəl əzbərləmək lazım deyil. Kompilyator C# mətnini işləyən formaya çevirən alətdir.

4 I/O və CPU üçün fərqli seçim

C#

```
// I/O ucun hazir async API-ni birbasa gozle.
string text = await http.GetStringAsync(url, ct);

// UI-de agir CPU isini ayrica planlamaq olar.
int value = await Task.Run(() => Calculate(), ct);
```

`http` HttpClient, `url` şəbəkə ünvanı, `ct` cancellation token-dir. `Calculate` sənin hesablama metodundur. Lambda `() => Calculate()` sonra icra ediləcək işi təqdim edir. **UI**, User Interface, istifadəçi interfeysidir. UI thread-i düymə və görünüş yeniləmələrini idarə edir; uzun CPU işi onu dondura bilər.

ASP.NET Core-da sadəcə bir sinxron baza çağırışını Task.Run ilə sarımaq onun I/O-sunu həqiqi asinxron etmir; ThreadPool thread-i yenə gözləyə bilər. Hazır async baza API-sini istifadə et. Serverdə ağır CPU işini əlavə Task.Run-a ötürmək ümumi hesablama gücünü artırmır. İşin həcmi böyükdürsə ayrıca worker, növbə və ya uyğun resurs planlaması lazım ola bilər.

`Task.Run(..., ct)` token-i iş hələ başlamayıbsa planlanmanı ləğv edə bilər. İşləyən `Calculate` token-i yoxlamırsa onu zorla dayandırmır. **Cooperative cancellation** əməliyyatın ləğv siqnalını özü nəzərə almasıdır.

5 Ardıcıl await və birlikdə başlatma

C#

```
// Ardicil: ikinci is birinci bitenden sonra baslayir.
var a = await GetPriceAsync(ct);
var b = await GetStockAsync(ct);

// Musteqil isler: evvel her ikisini baslat.
var priceTask = GetPriceAsync(ct);
var stockTask = GetStockAsync(ct);
await Task.WhenAll(priceTask, stockTask);

var price = await priceTask;
var stock = await stockTask;
```

`WhenAll` verilmiş bütün Task-lar tamamlananda tamamlanan Task qaytarır. Onları avtomatik yaratmır və mütləq ayrı CPU nüvələrində işlətdiyi mənasına gəlmir. Yuxarıda əməliyyatları metod çağırışları başlayır. İşlər bir-birindən müstəqil olmalıdır. İki 300 ms və 500 ms xarici gözləmə üçün ideal sadə modeldə ardıcıl vaxt 800 ms, üst-üstə düşən vaxt təxminən 500 ms ola bilər; real servis, şəbəkə və limitlər nəticəni dəyişir.

Eyni DbContext ilə iki paralel sorğunu WhenAll-a vermək düzgün deyil. DbContext eyni anda bir neçə əməliyyatı təhlükəsiz icra etmək üçün nəzərdə tutulmayıb. Ayrı context-lər istifadə edəndə də connection pool və baza resurslarını tükətməmək lazımdır. Connection pool əvvəl açılmış fiziki bağlantıları təkrar istifadə edən

mexanizmdir.

6 WhenAll xətası və WhenAny

WhenAll bütün verilmiş işlərin bitməsini gözləyir. Biri xəta atanda digərlərini avtomatik dayandırmır. Bir və ya daha çox fault varsa ümumi Task faulted olur. Fault yox, amma cancellation varsa canceled ola bilər. `await` çox vaxt bir exception-u yenidən atır; bütün xətalar ümumi Task-ın `Exception` məlumatında saxlanır.

C#

```
Task all = Task.WhenAll(tasks);
try
{
    await all;
}
catch
{
    if (all.Exception is not null)
        foreach (var error in all.Exception.InnerExceptions)
            Console.WriteLine(error.Message);
    throw;
}
```

`tasks` əvvəl yaradılmış Task kolleksiyasıdır. `Exception` xəta obyektidir. `InnerExceptions` toplanmış xətaları verir. `throw;` ilkin xəta izini qoruyaraq xətanı yuxarı ötürür. Cancellation da catch-ə düşə bilər, bu halda `Exception` null ola bilər; buna görə yoxlama var.

`WhenAny` ilk tamamlanan Task-ı verir. “Tamamlandı” uğur, fault və ya cancellation ola bilər. Qalib Task-ı ayrıca await etmək onun həqiqi nəticəsini və ya xətasını almaq üçündür. Qalan Task-lar avtomatik ləğv olunmur.

`WaitAsync(timeout)` kimi gözləməyə vaxt məhdudiyyəti qoymaq əməliyyatın özünü mütləq dayandırmır. Əsas əməliyyat cancellation dəstəkləyirsə token ona da ötürülməlidir. Timeout gözləmənin bitməsi, cancellation isə əməliyyata dayanma istəyidir; iki ayrı müqavilədir.

7 CancellationToken və vaxt sərhədi

`CancellationToken` yalnız siqnalı oxumaq üçündür. `CancellationTokenSource` siqnalı yaradır və ləğv edir. `CancelAfter` müəyyən müddətdən sonra ləğv istəyir. `ThrowIfCancellationRequested` istək varsa `OperationCanceledException` atır; bu, əməkdaşlıq edən kodda normal ləğv yoludur.

C#

```
using var timeout = new CancellationTokenSource();
timeout.CancelAfter(TimeSpan.FromSeconds(5));

using var linked = CancellationTokenSource
    .CreateLinkedTokenSource(userToken, timeout.Token);

string text = await http.GetStringAsync(url, linked.Token);
```

Linked token hər iki mənbədən biri ləğv istəyəndə siqnal verir. `userToken` istifadəçinin və ya HTTP sorğusunun token-i ola bilər. `TimeSpan` vaxt müddətidir. `using` source-un resurslarını iş bitəndə təmizləyir. Token-i aşağıdakı async metodlara ötürməsən siqnal zəncirin sonunda itə bilər. Baza sürücüsünün cancellation-u qəbul etməsi serverdə hər şeyin dərhal geri alındığına zəmanət vermir; əməliyyatın nəticəsi ayrıca nəzərə alınmalıdır.

8 Result Wait və deadlock

`.Result` və `.Wait()` Task bitənə qədər çağıran thread-i bloklaya bilər. **Deadlock** tərəflərin bir-birini gözləyib irəliləyə bilməməsidir. UI-də metod `.Result` ilə bloklanır, həmin metodun await-dən sonrakı davamı UI thread-də işləmək istəyirsə dairə yaranır: UI nəticəni, nəticə UI-ni gözləyir.

SynchronizationContext davamın hansı mühitdə icra olunacağını bildirən abstraksiyadır. UI mühitində bu çox vaxt UI thread-idir. ASP.NET Core-da standart olaraq klassik request SynchronizationContext yoxdur. Buna görə UI-dəki həmin konkret deadlock mexanizmi eyni formada gözlənilmir, amma blocking yenə ThreadPool starvation yarada bilər. **Starvation** işləməyə hazır işlərin thread tapmaqda uzun gözləməsidir. Digər kilid deadlock-ları hər mühitdə mümkündür.

`ConfigureAwait(false)` həmin await üçün uyğun context-i tutub davamı ora qaytarmaq tələbini azaldır. “Mütləq yeni thread-də işlə”, “bütün deadlock-ları həll et” və “ExecutionContext-i sil” demək deyil. **ExecutionContext** AsyncLocal kimi məntiqi çağırış məlumatlarının axın mexanizmidir. Kitabxana kodunda context tələb olunmursa faydalı ola bilər; UI görünüşünü dəyişən kodda thread tələbi qorunmalıdır. ASP.NET Core tətbiq kodunda onu hər sətirə yazmaq universal məcburiyyət deyil.

9 Async void və resurs ömrü

`async void` çağırana await edilən Task vermir. Ona görə tamamlanmanı və xətanı adi Task kimi izləmək olmur. Əsas istisna void imzalı event handler-dir: düymə hadisəsi kimi. Digər iş metodları üçün `Task` və ya `Task<T>` daha uyğun müqavilədir.

Fire and forget işi başlayıb tamamlanmasını izləməməkdir. HTTP sorğusunda `_ = SendAsync()` yazmaq həmin işin etibarlı tamamlanmasına zəmanət vermir. Sorğu scope-u bağlana, istifadə olunan DbContext dispose oluna və xəta müşahidəsiz qala bilər. Etibarlı fon işi üçün host-un idarə etdiyi worker, öz scope-u və davamlı iş tələb olunursa növbə lazımdır.

C#

```
// Resurs await tamamlanana qeder aciq qalir.
static async Task<string> ReadAsync(string path)
{
    using var reader = new StreamReader(path);
    return await reader.ReadToEndAsync();
}
```

`StreamReader` fayldan mətn oxuyur. `using` reader-i metoddan çıxanda təmizləyir. Bu nümunədə await əhəmiyyətlidir: async işi bitmədən scope-dan çıxıb resursu bağlamaq olmaz. **Dispose** fayl kimi resursları buraxır; managed yaddaşı dərhal silmir.

10 ValueTask nə vaxt lazımdır

`ValueTask<T>` nəticəni birbaşa, Task vasitəsilə və ya xüsusi mənbə ilə təmsil edə bilən value type-dır. Tez-tez sinxron tamamlanan yüksək tezlikli əməliyyatlarda bəzi Task allocation-larını azalda bilər. **Allocation** yeni obyekt üçün yaddaş ayırmaqdır. Task üçün hazır cached, yəni təkrar istifadə olunan tamamlanmış nəticələr də ola bilər; “hər Task çağırışı mütləq yeni heap obyekti yaradır” doğru deyil.

ValueTask value type olsa da “həmişə stack-dədir” deyil. Sinfin sahəsində və async state machine daxilində saxlanması kimi hallar var. **Stack** icra çağırışları üçün yaddaş sahəsi, **heap** obyektlərin idarə olunan yaddaş sahəsidir; saxlanma yeri təkcə tipin adından müəyyən edilmir.

Ümumi API-lərdə başlanğıc seçim Task-dır. ValueTask seçmək istehlakçı qaydalarını artırır: ümumi qayda kimi bir dəfə await et, saxlayıb dəfələrlə gözləmə. Bir neçə dəfə gözləmək lazımdırsa bir dəfə `AsTask()` ilə Task-a çevirib onu istifadə et. Dəqiq məhdudiyyət mənbə implementasiyasından asılıdır; təhlükəsiz ümumi müqaviləyə əməl etmək lazımdır. Ölçmə olmadan “ValueTask daha yenidir, deməli hər yerdə yaxşıdır” nəticəsi çıxarma.

11 Race condition və məhdud paralellik

Race condition nəticənin əməliyyatların nəzarətsiz gəliş sırasından asılı olmasıdır. `counter++` oxu, artır, yaz addımlarından ibarət olduğundan iki thread eyni ilkin dəyəri oxuyub bir artımı itirə bilər. `Interlocked.Increment` bir artımı digər paralel artımlara qarşı bölünməz edir. Bu **atomic operation** anlayışıdır; baza transaction-unun bütün biznes işi üçün atomikliyi ilə eyni ölçüdə deyil.

Lock kritik hissəyə eyni anda məhdud girişi təşkil edir. Adi `lock` bloku içində await istifadə edilmir. Asinxron gözləmə ilə giriş məhdudlaşdırmaq üçün `SemaphoreSlim.WaitAsync` uyğun ola bilər. Semaphore müəyyən sayda paralel icazə verir. `Release` adətən `finally` daxilində çağırılır ki, xəta olanda da icazə geri qaytarılsın.

C#

```
await Parallel.ForEachAsync(
    ids,
    new ParallelOptions
    {
        MaxDegreeOfParallelism = 4,
        CancellationToken = ct
    },
    async (id, token) =>
    {
        await FetchAsync(id, token);
    });
```

`ids` işlənəcək identifikatorlar, `FetchAsync` sənin I/O metodundur. Maksimum 4 paralel iş resurs yükünü məhdudlaşdırır. Bu rəqəm sadəcə nümunədir; servis limitinə və connection pool-a görə ölçülməlidir. `Parallel.ForEach`-ə `async` lambda vermək async void tipli səhvlərə səbəb ola bilər; asinxron iş üçün uyğun API və müqavilə seç.

12 Müsahibə cavabları və məşq

Await hansı thread-də davam edir? Mühit, context, scheduler və əməliyyatın artıq tamamlanıb tamamlanmamasından asılıdır. Eyni thread-ə zəmanət yoxdur; həmişə başqa thread-ə keçid də yoxdur.

Async proqramı sürətləndirirmi? I/O gözləməsində thread resursunu sərbəst saxlaya bilər və müstəqil işləri üst-üstə salmaq olar. CPU işini avtomatik sürətləndirmir. Daha çox sorğuya xidmətlə bir sorğunun cavab vaxtını ayırmaq lazımdır.

Məşq. İki müstəqil `Task.Delay`, biri 300 ms, o biri 500 ms olsun. Əvvəl ardıcıl await et, sonra əvvəl hər ikisini çağırıb WhenAll gözlə. Vaxtı `Stopwatch` ilə ölç; o, keçən vaxtı ölçən .NET sinfidir. Sonra bir işdə xəta yarat: digərinin avtomatik dayanmadığını gör. Nəhayət shared sayğacı paralel artırıb adi ++ ilə Interlocked fərqini müşahidə et. Vaxt rəqəmləri yükə görə dəyişəcək.

13 Rəsmi mənbələr

- Microsoft Learn - Asynchronous programming
- Microsoft Learn - Async scenarios
- Microsoft Learn - Task based Asynchronous Pattern
- Microsoft Learn - Task.WhenAll
- Stephen Toub - ConfigureAwait FAQ
- Stephen Toub - Understanding ValueTask

Yoxlama tarixi 28 oktyabr 2026. C# nümunələri tədris fraqmentləridir; şəbəkə və baza çağırışları real servisdə icra edilməyib.